

Laboratorio ARSW No.6

María Belén Quintero Aldana

Nikolas Martinez Rivera

1. Redux avanzado

- Agrega estados loading/error por *thunk* y muéstralos en la UI.

Para agregar estos estados fue necesario cambiar el initialState del slice que se encontraba en blueprintsSlice.js.

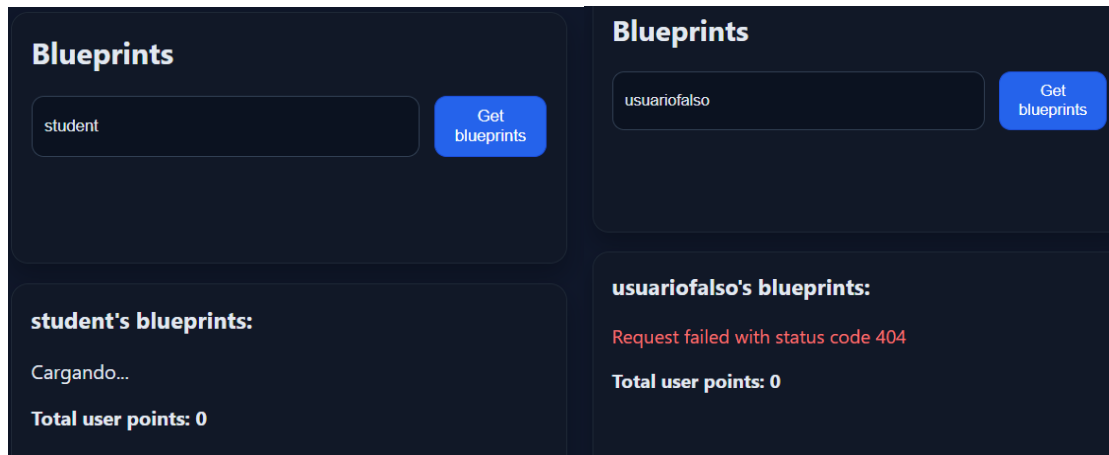
```
initialState:
{
  authors: { data: [], status: 'idle', error: null },
  byAuthor: { data: {}, status: 'idle', error: null },
  current: { data: null, status: 'idle', error: null },
},
```

Como se puede ver cambio a una estructura con un mejor manejo de status y de error, antes con la anterior manera en la que declaraba initialState se sobrescribían los estados y se perdía información en la operación que se estaba realizando, por lo tanto, se separa para que cada operación tenga su data, status y error por consecuencia se pueden mostrar los estados solicitados de loading/error por separado en la UI. Con este cambio también se modificó la sección de extraReducers, allí se añadieron los estados solicitados y se modificaron los anteriores debido al cambio en el initialState. Por cada thunk creado con createAsyncThunk se generan automáticamente tres acciones pending, fulfilled y rejected. En extraReducers se captura cada una para actualizar el estado correspondiente de esa operación, permitiendo mostrar estados de carga y error de forma independiente por operación en la UI como se mencionó anteriormente que era el objetivo

Con esos cambios realizados en el blueprintsSlice.js se procedió a modificar la blueprintsPage.jsx en la cual se agregó lo siguiente.

```
{byAuthor.status === 'loading' && <p>Cargando...</p>}
{byAuthor.error && <p style={{ color: '#f87171' }}>{byAuthor.error}</p>}
{!items.length && byAuthor.status !== 'loading' && !byAuthor.error && <p>Sin resultados.</p>}
```

Estas líneas hacen posible que el estado sea visible en la UI pues esto conecta lo que pasa en el slice y muestra lo que pasa con los thunk.



Las anteriores capturas muestran como se ve en la UI los estados.

- Implementa *memo selectors* para derivar el top-5 de blueprints por cantidad de puntos.

Para la implementacion de memo selector la cual quiere decir que son selectors que tienen memoria (por eso el memo) se hizo lo siguiente.

```
export const selectTop5Blueprints = createSelector(
  (state) => state.blueprints.byAuthor.data,
  (byAuthorData) => {
    const allBlueprints = Object.values(byAuthorData).flatMap(items => items)
    return allBlueprints
      .sort((a, b) => b.points.length - a.points.length)
      .slice(0, 5)
  }
)
```

Este fue declarado en el slice y como se puede ver se hace uso de `createSelector` que hace parte del toolkit de `redux`, este nos deja guardar los últimos inputs recibidos y el ultimo resultado calculado, ósea que si la entrada no cambia entonces esto va a devolver el resultado guardado sin rederizar y hacer operaciones de nuevo.

```

<div className="card">
  <thead>
    <tr>
      <th
        style={{
          textAlign: 'left',
          padding: '10px',
        }}
      >
        Top 5 blueprints of all users queried
      </th>
    </tr>
  </thead>
  <div style={{ display: 'flex', gap: 8, flexWrap: 'wrap', marginTop: 8 }}>
    {top5.map((bp) => (
      <div key={bp.name} style={{
        borderRadius: 6,
        padding: '6px 12px',
        fontSize: 13
      }}>
        <strong>{bp.name}</strong> { bp.points?.length || 0} pts
      </div>
    ))}
  </div>
</div>

```

además, el anterior código se agregó en el blueprintsPage para la visualización de ese memo selector en la UI.

2. Rutas protegidas

- Crea un componente `<PrivateRoute>` y protege la creación/edición.

Primero la creación del componente se puede ver a continuación.

```

import { Navigate } from 'react-router-dom'

export default function PrivateRoute({ children }) {
  const token = localStorage.getItem('token')
  return token ? children : <Navigate to="/login" />
}

```

Este componente como se puede verificar protege de la creación/edición, redirigiendo a los usuarios no autenticados a login. Este componente fue utilizado en el app.jsx en el cual se protegio la ruta de canvas

```

<Route path="/canvas" element={
  <PrivateRoute>
    <CanvasPage />
  </PrivateRoute>
} />

```

3. CRUD completo

- Implementa PUT

/api/blueprints/{author}/{name} y DELETE ... en el slice y en la UI.

Antes de la implementación del put nos pareció prudente implementar el DELETE en el backend para practicidad de concentrarnos en el front. Como implementaciones extra se añadió un endpoint con un DeleteMapping en el BlueprintsController.

```
@Operation(summary = "Elimina un blueprint") no usages new *
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Plano eliminado exitosamente"),
    @ApiResponse(responseCode = "404", description = "No se encontró el plano especificado")
})
@DeleteMapping("/{author}/{bpname}")
public ResponseEntity<> delete(@PathVariable String author, @PathVariable String bpname) {
    try {
        services.deleteBlueprint(author, bpname);
        return ResponseEntity.ok(new ApiResponseS<>{ code: 200, message: "Plano eliminado exitosamente", data: null});
    } catch (BlueprintNotFoundException e) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(new ApiResponseS<>{ code: 404, e.getMessage(), data: null});
    }
}
```

Como la opción de delete no estaba implementada desde la persistencia, es por eso por lo que se implementó en la interfaz de persistence y se añadió el servicio al BlueprintsServices. Y ya que estamos haciendo delete, en el front se añadió el thunk y el extraReducer case.

```
export const deleteBlueprint = createAsyncThunk('blueprints/deleteBlueprint', async ({ author, name }) => {
    await api.delete(`/blueprints/${encodeURIComponent(author)}/${encodeURIComponent(name)}`)
    return { author, name }
})
```

```
.addCase(deleteBlueprint.fulfilled, (s, a) => {
    const { author, name } = a.payload
    if (s.byAuthor.data[author]) {
        s.byAuthor.data[author] = s.byAuthor.data[author].filter(bp => bp.name !== name)
    }
})
```

Para mostrar el delete se añadió un botón dentro del tbody.

Con el delete ya implementado, solo nos haría falta el put, para esto empezamos añadiendo en el slice el thunk y el extraReducer, es importante mencionar que para esto utilizamos el endpoint que ya existía de PUT /{author}/{bpname}/points.

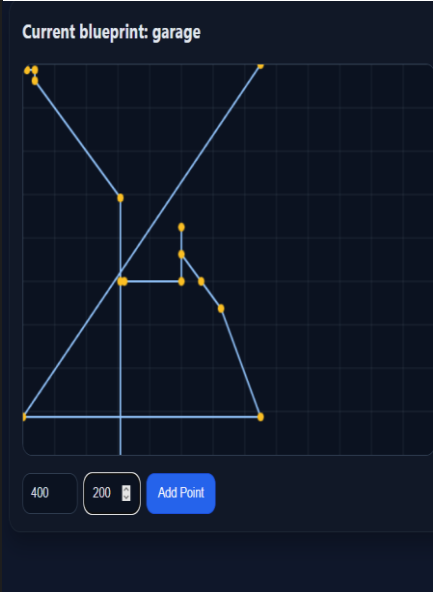
```
export const addPoint = createAsyncThunk('blueprints/addPoint', async ({ author, name, x, y }) => {
  await api.put(`/blueprints/${encodeURIComponent(author)}/${encodeURIComponent(name)}/points`, { x, y })
  return { author, name, x, y }
})
```

```
.addCase(addPoint.fulfilled, (s, a) => {
  const { author, name, x, y } = a.payload
  if (s.byAuthor.data[author]) {
    const bp = s.byAuthor.data[author].find(b => b.name === name)
    if (bp) bp.points.push({ x, y })
  }
  if (s.current.data?.author === author && s.current.data?.name === name) {
    s.current.data.points.push({ x, y })
  }
})
},
```

La manera en que se decidió mostrar la opción de añadir un punto en el blueprint fue a través de un input X y Y con un botón que confirma añadir en esa coordenada, el código y la respectiva visualización en UI es la siguiente.

```
<section className="card">
<h3 style={{ marginTop: 0 }}>Current blueprint: {current.data?.name || '...'}</h3>
<BlueprintCanvas points={current.data?.points || []} />

{current.data && (
  <div style={{ display: 'flex', gap: 8, marginTop: 12 }}>
    <input
      type="number"
      placeholder="x"
      className="input"
      style={{ width: 70 }}
      value={pointInput.x}
      onChange={(e) => setPointInput(p => ({ ...p, x: e.target.value })})
    />
    <input
      type="number"
      placeholder="y"
      className="input"
      style={{ width: 70 }}
      value={pointInput.y}
      onChange={(e) => setPointInput(p => ({ ...p, y: e.target.value })})
    />
    <button className="btn primary"
      onClick={() => dispatch(addPoint({
        author: current.data.author,
        name: current.data.name,
        x: Number(pointInput.x),
        y: Number(pointInput.y)
      }))}
    />
    Add Point
  </div>
)}
```



Como se puede ver en el código se aprovechó la tarjeta en la que estaba puesto el componente del Canvas para añadir los botones.

- Optimistic updates (revertir si falla).

Para estas actualizaciones optimistas en las cuales esperamos que el servidor responda de manera exitosa añadimos al slice 2 extraReducers.

```

.addCase(addPoint.pending, (s, a) => {
  const { author, name, x, y } = a.meta.arg
  if (s.byAuthor.data[author]) {
    const bp = s.byAuthor.data[author].find(b => b.name === name)
    if (bp) bp.points.push({ x, y })
  }
  if (s.current.data?.author === author && s.current.data?.name === name) {
    s.current.data.points.push({ x, y })
  }
})
.addCase(addPoint.rejected, (s, a) => {
  const { author, name, x, y } = a.meta.arg
  if (s.byAuthor.data[author]) {
    const bp = s.byAuthor.data[author].find(b => b.name === name)
    if (bp) bp.points = bp.points.filter(p => !(p.x === x && p.y === y))
  }
  if (s.current.data?.author === author && s.current.data?.name === name) {
    s.current.data.points = s.current.data.points.filter(p => !(p.x === x && p.y === y))
  }
})
},
)

```

Como se puede ver añadimos uno para el estado de pendiente y de rechazado, en el estado de pendiente es donde tenemos nuestra optimistic update por eso a diferencia del caso donde se es fulfilled no se utiliza el payload. Ahora, en caso de que no conteste el servidor como esperábamos y se rechace la acción es la razón por la cual tenemos el caso de rejected en el cual reversamos la actualización hecha en el componente.

4. Dibujo interactivo

- Reemplaza el svg por un lienzo donde el usuario haga *click* para agregar puntos.

Para este punto el svg fue reemplazado por el lienzo interactivo en el canvas, para hacer este cambio se hizo un nuevo componente que como tal reciclaba gran parte del código del lienzo original, la diferencia es que en este caso captamos los clicks en react lo cual se hace con `handleclick`, la manera en la que fue usado en esta situación fue de la siguiente manera.

```
const handleClick = (e) => {
  const rect = ref.current.getBoundingClientRect()
  const x = Math.round((e.clientX - rect.left) / scale)
  const y = Math.round((e.clientY - rect.top) / scale)
  const newPoints = [...points, { x, y }]
  setPoints(newPoints)
  onPointAdded(newPoints)
}
```

Aca en base al click realizado en el canvas se obtiene la posición y dimensiones usando `getBoundingClientRect()`, luego se calculan las coordenadas relativas de ese click al canvas esto restando `rect.left` y `rect.top` a `e.clientX` y `e.clientY`, y las ajusta según el `scale`. Con lo anterior se crea un nuevo arreglo de puntos agregando el punto `{x, y}` al arreglo existente `points`, actualiza el estado con `setPoints(newPoints)` y finalmente llama a `onPointAdded(newPoints)` para notificar que se agregó un nuevo punto. Esta fue la parte más relevante de código realizada para lo que este punto.

- Botón “Guardar” que envíe el blueprint.

Ahora el botón de guardar usa el estado de drawable canvas que es el `onPoints Added`, cuando lo presionamos llamamos a `handlesave`.

```
const handlesave = () => {
  if (!bpName) return alert('Ingresa un nombre')
  dispatch(createBlueprint({ author: username, name: bpName, points: drawnPoints }))
  setBpName('')
  setDrawnPoints([])
}
```

Es importante resaltar que para esto con el ánimo de identificar el blueprint creado, solicitamos un nombre para el blueprint.

5. Errores y *Retry*

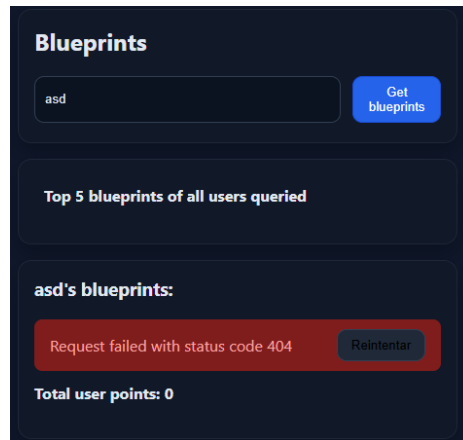
- Si GET falla, muestra un banner y un botón **Reintentar** que dispare el thunk.

Debido a que ya habíamos manejado estados de error para este punto solo fue necesario modificar la manera en cómo se mostraba el error y acompañarlo del `retry` solicitado.

```

<div style={{ background: '#7f1d1d', padding: '10px 16px', borderRadius: 8, display: 'flex' }}>
  <span style={{ color: '#fca5a5' }}>{byAuthor.error}</span>
  <button className="btn" onClick={() => dispatch(fetchByAuthor(selectedAuthor))}>
    Reintentar
  </button>
</div>

```



6. TESTS Y CI

Las pruebas ya se encontraban implementadas, solo fue necesario modificar pequeñas partes debido al cambio hecho en el primer punto, a continuación, se muestra la captura de pantalla con las pruebas pasadas después de un push.

All workflows				
Showing runs from all workflows				
4 workflow runs Event Status Branch Actor				
Update README with routes and backend disclaimer node-ci #4: Commit 266f149 pushed by NikoMAR3	main	5 minutes ago	20s	...
refactor: Improve README.md formatting adding emojis and cute things node-ci #3: Commit d6bd049 pushed by mboual	main	43 minutes ago	16s	...
refactor: Update README.md. Making even prettier node-ci #2: Commit 40294f5 pushed by mboual	main	Today at 6:34 PM	19s	...
fix: Test ajustados al codigo node-ci #1: Commit 1a05626 pushed by NikoMAR3	main	Mar 10, 10:59 PM GMT-5	21s	...

7.Mock y client

Se implementaron dos servicios con la misma interfaz (getAll, getByAuthor, getByAuthorAndName, create, addPoint, deleteBlueprint): apimock.js retorna datos en memoria sin hacer ninguna petición HTTP, útil para desarrollo y pruebas sin depender del backend, mientras que apiClient.js consume el API REST real usando Axios con autenticación

JWT. El módulo `blueprintsService.js` actúa como switch entre ambos según la variable de entorno `VITE_USE MOCK` en `.env`, permitiendo cambiar entre mock y cliente real con una sola línea de configuración tal como fue solicitado en el readme.